

Non-negative matrix factorisation

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

In this notebook, we exemplify **non-negative matrix factorisation** (NMF) by applying it to images of hand-written digits and finally to the matrices in Problem 3 of Problem Sheet - Week 3.

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import numpy.typing as npt
import torch
from typing import Optional

plt.style.use("ggplot")
```

Two basic implementations of NMF

The following two classes implement NMF via maximising the quasi-likelihood objective $L(W, H)$ using PyTorch and NumPy, respectively. By default, the PyTorch implementation uses GPU if available and switches to CPU as a fallback. The device type can also be chosen manually through the optional `device` argument of the constructor.

```
In [2]: class NMF_torch:
    def __init__(self, device: Optional[torch.device] = None) -> None:
        self._device = (
            torch.device(
                "cuda"
                if torch.cuda.is_available()
                else "mps" if torch.backends.mps.is_available() else "cpu"
            )
            if device is None
            else device
        )

    @property
    def device_type(self) -> str:
        return self._device.type

    def train(self, X: npt.NDArray[np.number], q: int, n_iter: int = 100) -> None:
        d, n = X.shape

        W = torch.rand(d, q, device=self._device, requires_grad=False)
        H = 1 / q * torch.rand(q, n, device=self._device, requires_grad=False)
        WH = W @ H
        Xc = torch.from_numpy(X).to(self._device, dtype=W.dtype).requires_grad_(False)

        logL = torch.empty(n_iter)
        dist = torch.empty(n_iter)

        for a in range(n_iter):
            W *= ((Xc / WH) @ H.T) / H.sum(dim=1).unsqueeze(0)
            WH = W @ H

            H *= (W.T @ (Xc / WH)) / W.sum(dim=0).unsqueeze(1)
            WH = W @ H

            logL[a] = torch.sum(Xc * torch.log(WH) - WH)
            dist[a] = torch.sum((Xc - WH) ** 2)

        self.W = W.cpu().numpy()
        self.H = H.cpu().numpy()
        self.WH = WH.cpu().numpy()
        self.logL = logL.cpu().numpy()
        self.dist = dist.cpu().numpy()
```

```
In [3]: class NMF_numpy:
    def train(self, X: npt.NDArray[np.number], q: int, n_iter: int = 100) -> None:
        d, n = X.shape

        self.W = np.random.uniform(low=0, high=1, size=(d, q)).astype(X.dtype)
        self.H = np.random.uniform(low=0, high=1 / q, size=(q, n)).astype(X.dtype)
        self.WH = self.W @ self.H

        self.logL = np.empty(n_iter, dtype=X.dtype)
        self.dist = np.empty(n_iter, dtype=X.dtype)

        for a in range(n_iter):
            self.W *= ((X / self.WH) @ self.H.T) / self.H.sum(axis=1)[None, :]
            self.WH = self.W @ self.H

            self.H *= (self.W.T @ (X / self.WH)) / self.W.sum(axis=0)[:, None]
            self.WH = self.W @ self.H

            self.logL[a] = np.sum(X * np.log(self.WH) - self.WH)
            self.dist[a] = np.sum((X - self.WH) ** 2)
```

In both cases the attributes `W`, `H` and `WH` of the class hold the final values of W and H and WH as NumPy arrays, respectively, while the attributes `logL` and `dist` are vectors that describe how $L(W, H)$ and $\|X - WH\|^2$, respectively, develop during the iteration.

Images of hand-written digits

Let us first load the data (which we studied in the PCA example) and select images of threes:

```
In [4]: zip_train = np.loadtxt("zip.train", dtype=np.float32)
        threes = zip_train[zip_train[:, 0] == 3]
        threes = threes[:, 1:]
```

Let us apply NMF to the images with $q = 2^3 = 8$. We adjust the pixel values (originally in $[-1, 1]$) upwards by 1.01, as the algorithm can have problems with zero values, and multiply by 0.5. We use the PyTorch implementation and also the NumPy implementation to compare performance:

```
In [5]: q = 2**3
        n_iter = 1000

        nmf = NMF_torch(torch.device("cpu"))
        nmf.train(0.5 * (threes + 1.01), q, n_iter)
```

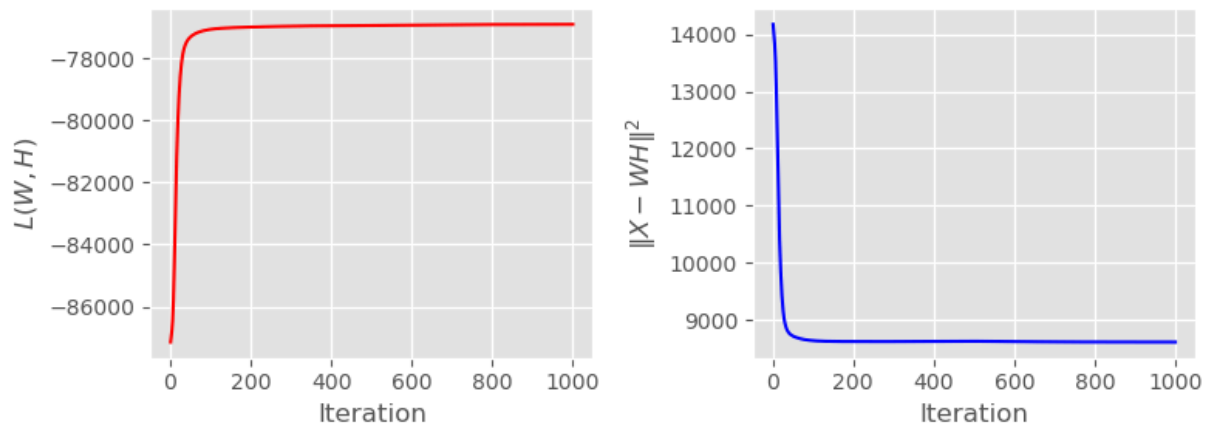
```
In [6]: nmf_comp = NMF_numpy()
        nmf_comp.train(0.5 * (threes + 1.01), q, n_iter)
```

Convergence:

```
In [7]: fig, axs = plt.subplots(1, 2, figsize=(8, 3))

        axs[0].plot(range(1, n_iter + 1), nmf.logL, color="red")
        axs[0].set_xlabel("Iteration")
        axs[0].set_ylabel(r"$L(W,H)$")
        axs[1].plot(range(1, n_iter + 1), nmf.dist, color="blue")
        axs[1].set_xlabel("Iteration")
        axs[1].set_ylabel(r"$\|X-WH\|^2$")

        plt.tight_layout()
        plt.show()
```



Then plot the NMF reconstructions of the first 9 images:

```
In [8]: threes_recons = 2.0 * nmf.WH - 1.01

def draw_img(ax, x):
    img = (1 - x) / 2
    img = img.reshape(16, 16)
    ax.imshow(img, cmap="gray", vmin=0, vmax=1)
    ax.axis("off")

fig, axes = plt.subplots(3, 3, figsize=(5, 5))

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes_recons[i,] - 0.01)

plt.tight_layout()
plt.show()
```



Recall that the input size is $16^2 = 256$. Let us increase q to $2^6 = 64$:

```
In [9]: q = 2**6
n_iter = 2000

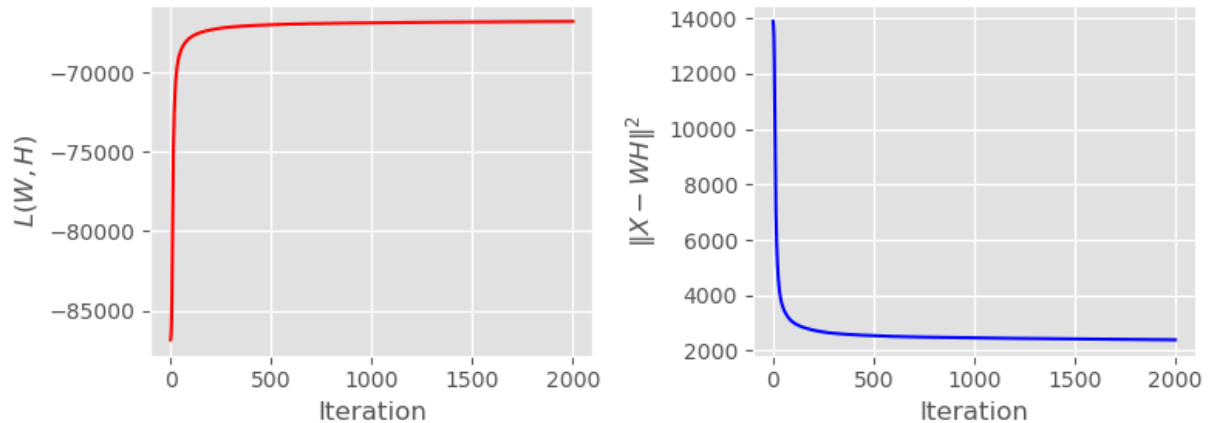
nmf.train(0.5 * (threes + 1.01), q, n_iter)
```

Convergence:

```
In [10]: fig, axs = plt.subplots(1, 2, figsize=(8, 3))

axs[0].plot(range(1, n_iter + 1), nmf.logL, color="red")
axs[0].set_xlabel("Iteration")
axs[0].set_ylabel(r"$L(W,H)$")
axs[1].plot(range(1, n_iter + 1), nmf.dist, color="blue")
axs[1].set_xlabel("Iteration")
axs[1].set_ylabel(r"$\|X-WH\|^2$")

plt.tight_layout()
plt.show()
```



Reconstructions of the first 9 images:

```
In [11]: threes_recons = 2.0 * nmf.WH - 1.01

fig, axes = plt.subplots(3, 3, figsize=(5, 5))

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes_recons[i,] - 0.01)

plt.tight_layout()
plt.show()
```

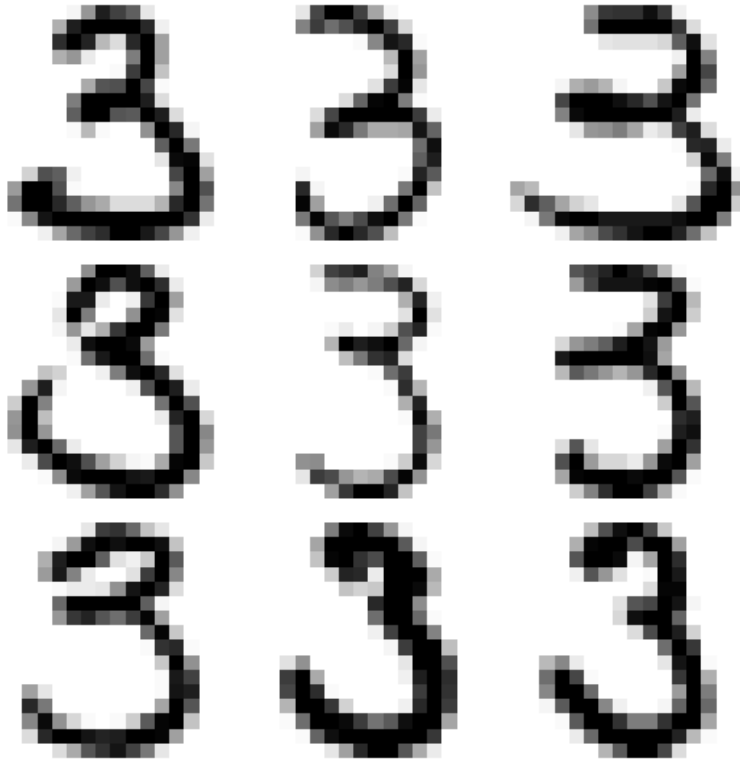


The originals look like this:

```
In [12]: fig, axes = plt.subplots(3, 3, figsize=(5, 5))

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes[i, :])

plt.tight_layout()
plt.show()
```



Matrices from Problem 3 in Problem Sheet - Week 3

Apply NMF to

$$X = I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

with $q = 1$:

```
In [13]: X = np.eye(2)
nmf = NMF_torch(torch.device("cpu"))
nmf.train(X, 1)
print("W = \n", nmf.W, "\n")
print("H = \n", nmf.H, "\n")
print("WH = \n", nmf.WH, "\n")
print("X = \n", X)
```

```
W =
[[3.3542743]
 [3.3542743]]
```

```
H =
[[0.14906354 0.14906354]]
```

```
WH =
[[0.5 0.5]
 [0.5 0.5]]
```

```
X =
[[1. 0.]
 [0. 1.]]
```

Finally, apply NMF to

$$X = I_2 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

with $q = 1$:

```
In [15]: X = np.ones((2, 2))
nmf = NMF_torch(torch.device("cpu"))
nmf.train(X, 1)
print("W = \n", nmf.W, "\n")
print("H = \n", nmf.H, "\n")
print("WH = \n", nmf.WH, "\n")
print("X = \n", X)
```

```
W =
[[4.101354]
 [4.101354]]
```

```
H =
[[0.2438219  0.24382192]]
```

```
WH =
[[1.  1.]
 [1.  1.]]
```

```
X =
[[1.  1.]
 [1.  1.]]
```

Perfect reconstruction: $X = WH$

But it is indeed not unique:

```
In [16]: nmf.train(X, 1)
print("W = \n", nmf.W, "\n")
print("H = \n", nmf.H, "\n")
print("WH = \n", nmf.WH)
```

```
W =
[[1.9100275]
 [1.9100276]]
```

```
H =
[[0.52355266  0.52355266]]
```

```
WH =
[[1.  1.]
 [1.  1.]]
```